

VLSI INTERNSHIP – TASK 4

VERILOG HDL IMPLEMENTATION OF FINITE STATE MACHINES

Submitted By : Likhith Gowda H R

Department of Electronics and Communication Engineering

Dayananda Sagar Academy of Technology and Management (DSATM)

Internship Domain : VLSI Design

Organization : Maincrafts Technology

July 2026

1. Introduction

Verilog HDL (Hardware Description Language) is widely used for designing and verifying sequential digital systems in FPGA and ASIC design flows. Finite State Machines (FSMs) are one of the most important sequential design methodologies used to model systems whose behavior depends on both the current state and input conditions. FSMs are extensively used in digital controllers, communication protocols, traffic signal systems, sequence detectors, and embedded applications.

In this task, FSM-based digital circuits including a **Moore FSM**, **Mealy FSM**, **Traffic Light Controller**, and **1011 Sequence Detector** were implemented using Verilog HDL. Functional verification was carried out using **Icarus Verilog**, and the generated waveforms were analyzed using **GTKWave** to validate the correctness of each design.

1.1 Objectives

- To understand Finite State Machine (FSM) design using Verilog HDL.
- To implement a Moore FSM.
- To implement a Mealy FSM.
- To design a Traffic Light Controller.
- To implement a 1011 Sequence Detector.
- To simulate FSM circuits using Icarus Verilog.
- To analyze waveforms using GTKWave.

1.2 Tools Used

- Verilog HDL
- Icarus Verilog
- GTKWave
- Ubuntu (WSL)

2. Moore FSM Implementation

2.1 Verilog Code

The following Verilog HDL code implements a three-state Moore Finite State Machine (FSM).

moore_fsm.v

```
module moore_fsm(  
    input clk,  
  
    input reset,  
  
    output reg y  
  
);  
  
parameter S0 = 2'b00,  
  
           S1 = 2'b01,  
  
           S2 = 2'b10;  
  
reg [1:0] state, next_state;  
  
always @(posedge clk or posedge reset)  
  
begin  
  
    if (reset)  
  
        state <= S0;  
  
    else  
  
        state <= next_state;  
  
end
```

```
always @(*)

begin

    case(state)

        S0: next_state = S1;

        S1: next_state = S2;

        S2: next_state = S0;

        default: next_state = S0;

    endcase

end

always @(*)

begin

    case(state)

        S0: y = 1'b0;

        S1: y = 1'b1;

        S2: y = 1'b0;

        default: y = 1'b0;

    endcase

end

endmodule
```

2.2 Testbench Code

The following testbench verifies the operation of the Moore FSM by generating clock and reset signals and observing the state transitions and output.

tb_moore_fsm.v

```
`timescale 1ns/1ps
module tb_moore_fsm;
    reg clk;
    reg reset;
    wire y;
    moore_fsm uut (
        .clk(clk),
        .reset(reset),
        .y(y)
    );

    always #5 clk = ~clk;

    initial
    begin
        $dumpfile("moore.vcd");
        $dumpvars(0, tb_moore_fsm);
    End

    initial
    begin
        clk = 0;
        reset = 1;
        #10;
        reset = 0;
```

```

        #60;

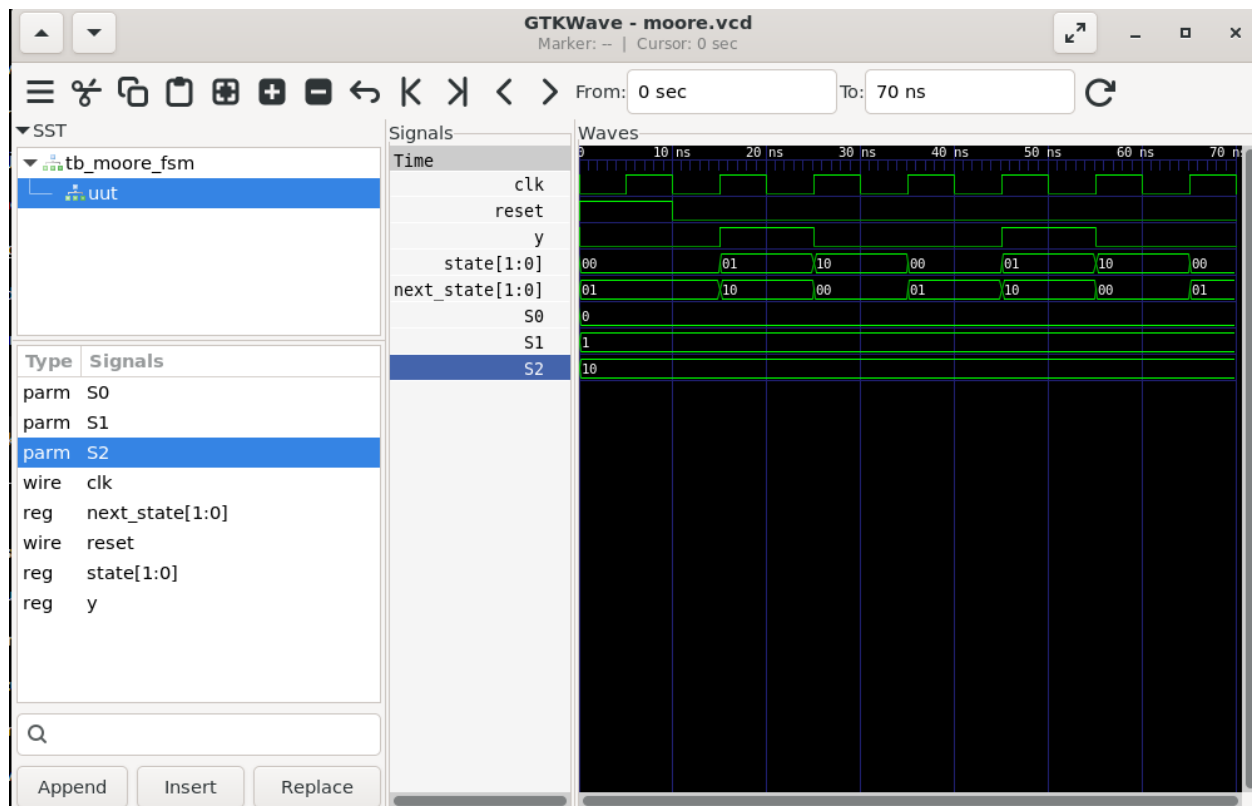
        $finish;

end

endmodule

```

2.3 Waveform Output



Observation:

The Moore FSM was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that the FSM transitions correctly through its predefined states. The output depends only on the current state, demonstrating the characteristic behavior of a Moore Finite State Machine.

3. Mealy FSM Implementation

3.1 Verilog Code

The following Verilog HDL code implements a Mealy Finite State Machine (FSM). The output depends on both the current state and the input.

mealy_fsm.v

```
module mealy_fsm(  
    input clk,  
    input reset,  
    input in,  
    output reg out  
);  
  
reg state;  
  
always @(posedge clk or posedge reset)  
begin  
    if (reset)  
    begin  
        state <= 0;  
        out <= 0;  
    end  
    else  
    begin  
        case(state)  
            0:  
                begin  
                    if(in)  
                        begin
```

```
        state <= 1;
        out <= 0;
    end
end

1:
begin
    if(in)
        begin
            out <= 1;
        end
    else
        begin
            state <= 0;
            out <= 0;
        end
    end
end

endcase

end

end
```

endmodule

3.2 Testbench Code

The following testbench verifies the operation of the Mealy FSM by applying different input combinations while generating clock and reset signals.

tb_mealy_fsm.v

```
`timescale 1ns/1ps

module tb_mealy_fsm;

    reg clk;

    reg reset;

    reg in;

    wire out;

    mealy_fsm uut (

        .clk(clk),

        .reset(reset),

        .in(in),

        .out(out)

    );

    always #5 clk = ~clk;

    initial

    begin

        $dumpfile("mealy.vcd");

        $dumpvars(0, tb_mealy_fsm);
```

```
end

initial

begin

    clk = 0;

    reset = 1;

    in = 0;

    #10 reset = 0;

    #10 in = 1;

    #10 in = 1;

    #10 in = 0;

    #10 in = 1;

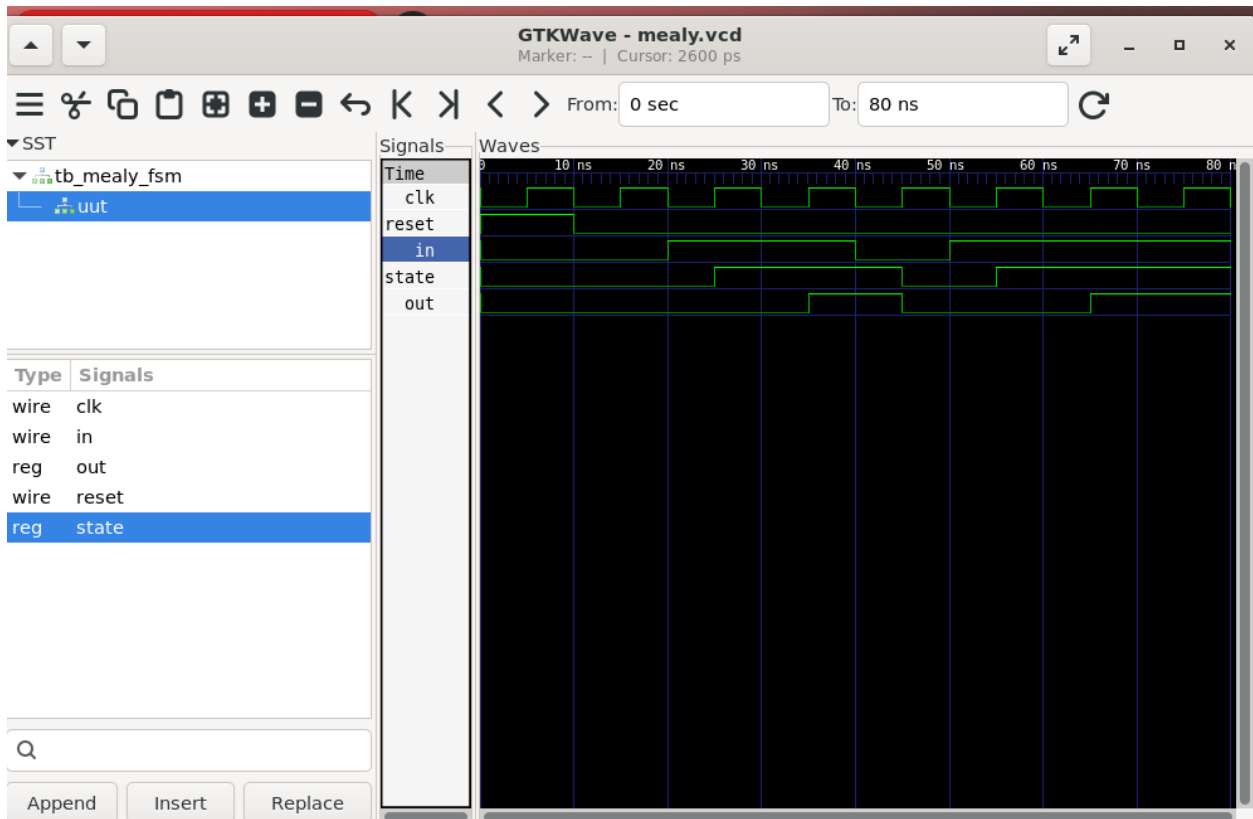
    #10 in = 1;

    #20 $finish;

end

endmodule
```

3.3 Waveform Output



Observation:

The Mealy FSM was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that the output changes according to both the current state and the input signal. The state transitions and output behavior validate the correct implementation of the Mealy Finite State Machine.

4. Traffic Light Controller Implementation

4.1 Verilog Code

The following Verilog HDL code implements a Traffic Light Controller using a Finite State Machine (FSM). The controller cycles through red, green and yellow states in a predefined sequence.

traffic_light.v

```
module traffic_light(  
    input clk,  
    input reset,  
    output reg [2:0] light  
);  
  
parameter RED      = 2'b00,  
           GREEN    = 2'b01,  
           YELLOW   = 2'b10;  
  
reg [1:0] state, next_state;  
  
always @(posedge clk or posedge reset)  
begin  
    if (reset)  
        state <= RED;  
    else  
        state <= next_state;  
end
```

```
always @(*)
begin
    case (state)
        RED:    next_state = GREEN;
        GREEN:  next_state = YELLOW;
        YELLOW: next_state = RED;
        default: next_state = RED;
    endcase
end

always @(*)
begin
    case (state)
        RED:    light = 3'b100;    // Red ON
        GREEN:  light = 3'b001;    // Green ON
        YELLOW: light = 3'b010;    // Yellow ON
        default: light = 3'b100;
    endcase
end

endmodule
```

4.2 Testbench Code

The following testbench verifies the operation of the Traffic Light Controller by generating clock and reset signals and observing the light output sequence through state transitions.

tb_traffic_light.v

```
`timescale 1ns/1ps

module tb_traffic_light;

reg clk;

reg reset;

wire [2:0] light;

traffic_light uut (

    .clk(clk),

    .reset(reset),

    .light(light)

);

always #5 clk = ~clk;

initial

begin

    $dumpfile("traffic_light.vcd");

    $dumpvars(0, tb_traffic_light);

end

initial

begin

    clk = 0;

    reset = 1;
```

```

        #10 reset = 0;

        #60;

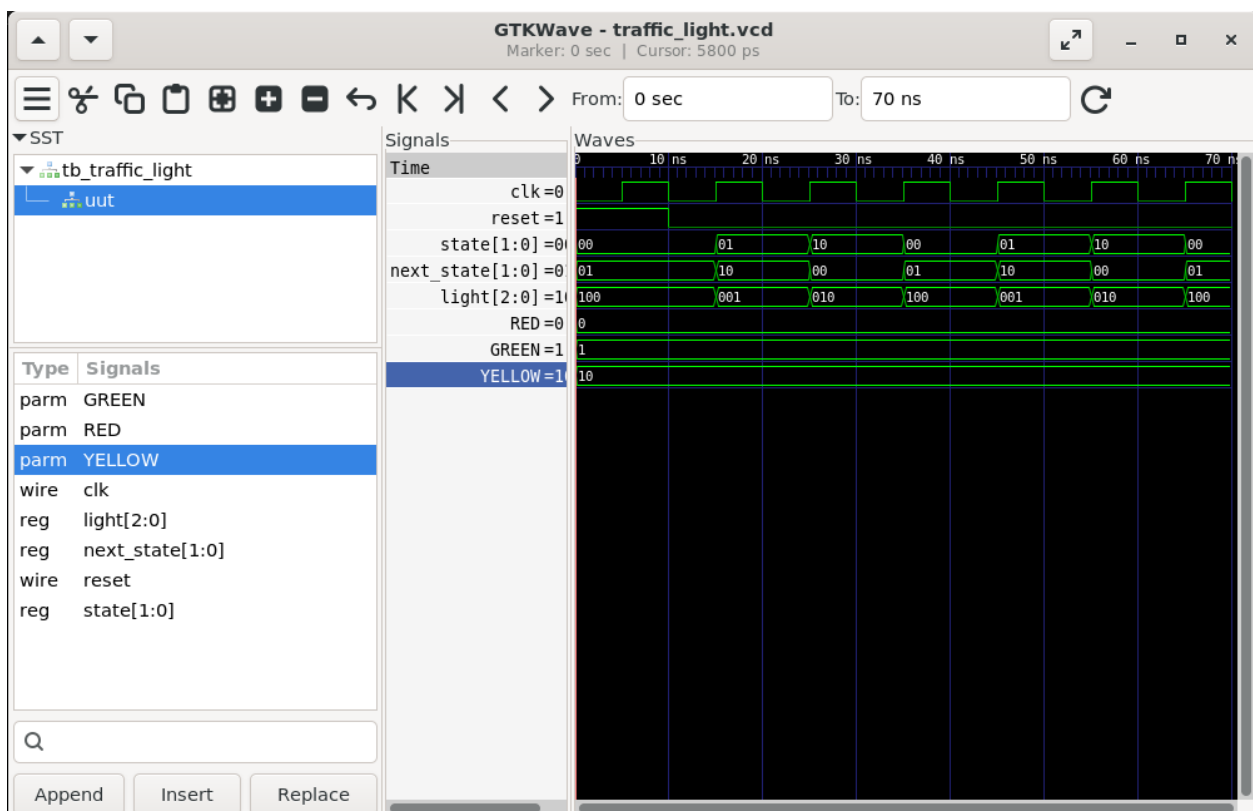
        $finish;

    end

endmodule

```

4.3 Waveform Output



Observation:

The Traffic Light Controller was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms the correct state transitions through **Red** → **Green** → **Yellow** → **Red**. The output signal changes according to the active state, validating the correct operation of the FSM-based traffic light controller.

5. Sequence Detector (1011) Implementation

5.1 Verilog Code

The following Verilog HDL code implements a Moore Finite State Machine (FSM) based Sequence Detector for detecting the binary sequence 1011. The detector asserts the output when the complete sequence is recognized.

sequence_detector.v

```
module sequence_detector(  
    input clk,  
    input reset,  
    input in,  
    output reg detected  
);  
  
parameter S0 = 3'd0,  
           S1 = 3'd1,  
           S2 = 3'd2,  
           S3 = 3'd3,  
           S4 = 3'd4;  
  
reg [2:0] state, next_state;  
  
always @(posedge clk or posedge reset)  
begin  
    if(reset)  
        state <= S0;  
    else  
        state <= next_state;
```

```
end

always @(*)
begin
    case(state)
        S0:
            if(in)
                next_state = S1;
            else
                next_state = S0;
        S1:
            if(in)
                next_state = S1;
            else
                next_state = S2;
        S2:
            if(in)
                next_state = S3;
            else
                next_state = S0;
        S3:
            if(in)
                next_state = S4;
            else
                next_state = S2;
    endcase
end
```

```
S4:
    if(in)
        next_state = S1;
    else
        next_state = S2;
    default:
        next_state = S0;
endcase
end
always @(*)
begin
    if(state == S4)
        detected = 1'b1;
    else
        detected = 1'b0;
    end
end
endmodule
```

5.2 Testbench Code

The following testbench verifies the operation of the Sequence Detector by applying different input bits. The binary sequence 1011 is applied to verify that the detector asserts the output after successfully recognizing the sequence.

tb_sequence_detector.v

```
`timescale 1ns/1ps

module tb_sequence_detector;

    reg clk;
    reg reset;
    reg in;
    wire detected;

    sequence_detector uut (
        .clk(clk),
        .reset(reset),
        .in(in),
        .detected(detected)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("sequence_detector.vcd");
        $dumpvars(0, tb_sequence_detector);
    end

    initial
    begin
        clk = 0;
        reset = 1;
        in = 0;
        #10 reset = 0;
```

```

#10 in = 1;

#10 in = 0;

#10 in = 1;

#10 in = 1;

#20 in = 0;

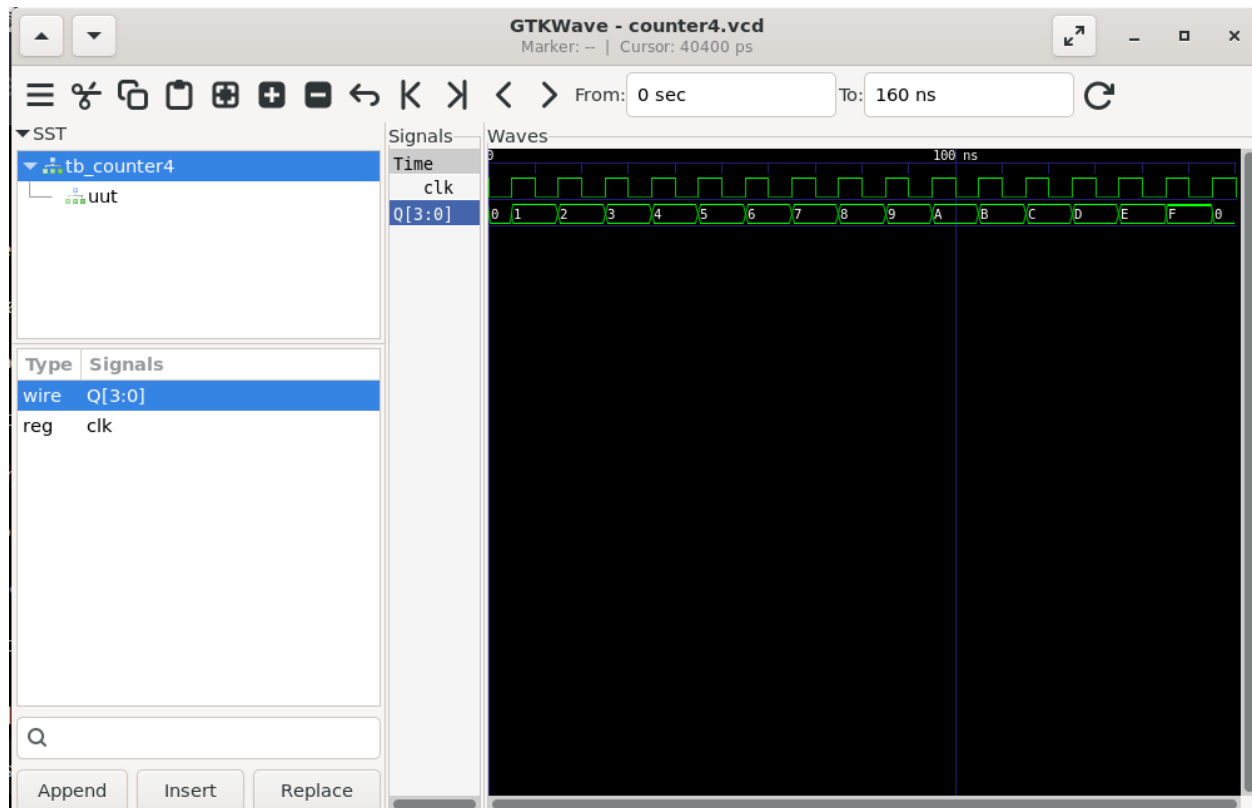
#20 $finish;

end

endmodule

```

5.3 Waveform Output



Observation:

The Sequence Detector was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that the FSM correctly detects the binary sequence **1011**. The detected output becomes HIGH only after the complete sequence is received, validating the correct operation of the sequence detector.

6. Results

The Moore FSM, Mealy FSM, Traffic Light Controller, and Sequence Detector were successfully implemented using Verilog HDL. Functional verification was performed using dedicated testbenches, and waveform analysis using GTKWave confirmed that all FSM-based circuits operated according to their expected functionality.

7. Applications

- FPGA Design
- ASIC Design
- Digital Control Units
- Traffic Signal Controllers
- Sequence Detection Systems
- Communication Protocol Controllers
- Embedded Systems
- VLSI Design and Verification

8. Conclusion

The implementation and verification of Moore FSM, Mealy FSM, Traffic Light Controller, and Sequence Detector using Verilog HDL were successfully completed. Simulation using Icarus Verilog and waveform analysis using GTKWave verified the correctness of all implemented designs. This task provided practical experience in finite state machine design, HDL coding, simulation, and verification, which are fundamental aspects of digital system and VLSI design.

■ References

1. Icarus Verilog Documentation
2. GTKWave User Guide
3. Verilog HDL – Samir Palnitkar
4. Digital Design – Morris Mano
5. IEEE Verilog HDL Standard